

Privacy Leakage and Lightweight Defenses in Graph Neural Networks for Sensitive Data: A Systematic Empirical Study

Krish Sharma and Dr. Shaikh Arifuzzaman

DiSC Lab, University of New Orleans

krishkiaan82@gmail.com

Abstract

Graph neural networks (GNNs) are increasingly used on sensitive relational data in healthcare, cybersecurity, and social networks. A key concern is whether these models leak *membership information*—that is, whether an attacker can tell if a specific person’s (or node’s) data was used to train the model. We present a systematic study of membership inference attacks (MIAs) against GNNs and non-graph baselines, under controlled graph structure and five lightweight defenses. We ran 420 experiments across six synthetic graph datasets (varying *homophily* and *density*), four model types, and five defense mechanisms, with five random seeds per configuration. Our main findings are: (1) GNNs are not uniformly more vulnerable than non-graph models; vulnerability depends on *graph structure* and *model type*—specifically, GCN leaks more when the graph has *low homophily* and is *sparse*, while GraphSAGE and baselines remain closer to random guessing. (2) *Homophily* and *density* strongly influence leakage: when the graph is “friendly” to learning (high homophily), all models leak little; when the graph is “unfriendly” (low homophily, sparse), GCN’s leakage rises. (3) *Label smoothing* often *increases* leakage and should not be used as a privacy defense; *edge sparsification* is the only defense that significantly reduced leakage in our tests (for GCN on low-homophily medium-density graphs). (4) Vulnerability is tied to *generalization*: when the model generalizes well, attack success is near chance; when it does not, both calibration error and leakage increase. We provide a reproducible, config-driven framework and recommend that practitioners avoid label smoothing for privacy and consider edge sparsification where structure is adverse.

Keywords: graph neural networks, membership inference, privacy, homophily, lightweight defenses, calibration.

1. Introduction

1.1 Motivation in Plain Language

Suppose a hospital trains a machine learning model on patient records to predict disease risk. Someone might try to ask: “Was *my* record used to train this

model?” If the model’s behavior betrays that answer, that is a *privacy leak*: it reveals membership in the training set. Such attacks are called **membership inference attacks (MIAs)**. They are well studied for models that work on tables or images, but less is known when the data are *graphs*—networks of connected entities (e.g., people linked by contacts, papers by citations). **Graph neural networks (GNNs)** are the standard tool for learning from such data. This project asks: *How vulnerable are GNNs to membership inference, and can simple, cheap defenses reduce that vulnerability without ruining the model’s usefulness?*

1.2 Research Questions

We address four questions:

- **RQ1:** Are GNNs more vulnerable to membership inference than simple non-graph models (logistic regression, MLP)?
- **RQ2:** How do two key properties of the graph—**homophily** and **density**—affect leakage? (These are defined in plain language below.)
- **RQ3:** Can lightweight defenses (DropEdge, label smoothing, early stopping, confidence masking, edge sparsification) reduce leakage without large loss in accuracy?
- **RQ4:** What is the tradeoff between *utility* (accuracy) and *privacy* (low attack success)?

1.3 Key Concepts Explained in Plain Language

Graph. A set of *nodes* (e.g., people, papers) and *edges* (links between them). Here we study *node classification*: each node has a label (e.g., topic of a paper), and the model predicts that label using the node’s own features and the pattern of who is connected to whom.

Homophily (in plain language). “Birds of a feather flock together.” **Homophily** is the extent to which *connected nodes tend to have the same label*.

- **High homophily:** Most links connect nodes that are alike (e.g., papers in the same field cite each other). The graph structure then “helps” the model: neighbors give a good hint about a node’s label.

- **Low homophily:** Many links connect nodes with *different* labels. The graph is noisier; the model has a harder time using links to predict correctly.

We measure homophily as the fraction of edges that connect two nodes with the *same* label (0 = no same-label links, 1 = all links same-label). In our experiments, “high” homophily is about 0.80 and “low” is about 0.30.

Density (in plain language). **Density** is how many edges the graph has compared to the maximum possible number of edges.

- **Sparse:** Few links; each node has few neighbors.

- **Dense:** Many links; each node has many neighbors.

We use three levels: sparse, medium, and dense (exact values in the Method section).

Membership inference attack (MIA). An attacker who can query the model tries to guess: “Was this node’s data in the training set?” The idea is that models often behave slightly differently on data they were trained on (e.g., slightly more confident) than on new data. The attacker uses that signal. We measure success with **attack AUC**: 0.5 means random guessing; 1.0 would mean perfect detection. So *higher attack AUC = more privacy leakage*.

Utility. How well the model does its main job (here, predicting node labels). We measure this with **accuracy** (and F1, AUROC). We want *high utility* and *low leakage*.

Calibration. Whether the model’s confidence matches reality (e.g., when it says “80% sure,” is it right about 80% of the time?). We measure **expected calibration error (ECE)**. Poor calibration (high ECE) often goes with higher leakage, because the attacker can exploit overconfident predictions.

2. Related Work (Brief)

Membership inference was introduced for standard classifiers by Shokri et al.; attackers use confidence scores or train a small “attack” model on top of the target model’s outputs. For GNNs, prior work has shown that graph structure can affect vulnerability and that some defenses (e.g., differential privacy) are strong but costly. We focus on *lightweight* defenses and on *systematically* varying graph structure (homophily and density) and model type, with rigorous comparison to non-graph baselines and statistical testing.

3. Methodology

3.1 Datasets

We use **synthetic graphs** so we can precisely control homophily and density. Each graph has 400 nodes, 50 features per node, and 5 classes. We generate six dataset types:

- **High homophily** (~ 0.80): sparse, medium, and dense.
- **Low homophily** (~ 0.30): sparse, medium, and dense.

Real citation networks (Cora, Citeseer) are supported in our codebase but were not included in the run reported here due to data download constraints; conclusions below therefore apply to these controlled synthetic settings.

3.2 Models

- **LogReg**: Logistic regression using only node features (no graph). Baseline.
- **MLP**: A small neural network on node features only (no graph). Baseline.

- **GCN:** Two-layer graph convolutional network; uses features and graph structure.
- **GraphSAGE:** Two-layer GraphSAGE; uses features and graph structure with sampling.

So we compare two *graph* models (GCN, GraphSAGE) to two *non-graph* models (LogReg, MLP) on the same data.

3.3 Attacks

We implement three attack evaluations:

1. **Confidence-based attack:** We build a small classifier that takes summary statistics of the model’s predictions (e.g., max probability, entropy) and predicts “member” vs “non-member.” This is our main privacy metric.
2. **Threshold attack:** A simple rule: if the model’s confidence on the true label is above a threshold, call it “member.” We report its AUC for comparison.
3. **Shadow-model attack:** We train an extra “shadow” model on a different split of data, then train the attack classifier on the shadow’s outputs and evaluate it on the target model’s outputs. This checks that our findings are not specific to one attack type.

Privacy metric: We report **attack AUC** (0.5 = no leakage, 1.0 = full leakage). We also report **calibration error (ECE)** on the test set.

3.4 Defenses (Lightweight)

- **None:** No defense.
- **DropEdge:** Randomly remove some edges during each training step.
- **Label smoothing:** Replace hard “0/1” labels with softer targets (e.g., 0.9 and 0.025) to reduce overconfidence.
- **Early stopping:** Stop training when validation loss stops improving.
- **Confidence masking:** At test time, only show the top few class probabilities (zero out the rest).
- **Edge sparsification:** Before training, remove a fraction of edges (e.g., those with low feature similarity).

Defenses are applied only to GNNs; baselines (LogReg, MLP) are always “none.”

3.5 Experimental Protocol

- For each combination of dataset, model, and defense, we run **5 random seeds**.
- We report **mean $\pm$ standard deviation** for accuracy, attack AUC, and ECE.
- We run **paired t-tests** (defense vs no defense, same seeds) and report p-values where relevant.

- Total: **420 runs** (6 datasets x 4 models x up to 6 defense configurations x 5 seeds, with baselines having only “none”).

4. Results and In-Depth Interpretation

4.1 RQ1: Are GNNs More Vulnerable Than Non-Graph Baselines?

Short answer: No—not always. It depends on the graph. When the graph is “friendly” (high homophily), no model leaks much. When the graph is “unfriendly” (low homophily, especially sparse), GCN leaks the most; GraphSAGE and the baselines stay closer to random.

In plain language:

On “easy” graphs where connected nodes tend to share the same label (high homophily), every model—logistic regression, MLP, GCN, and GraphSAGE—achieves high accuracy and the attacker’s success is close to a coin flip (attack AUC about 0.49–0.54). So in that setting, **GNNs are not more vulnerable** than the simple models.

On “hard” graphs where many links connect nodes with *different* labels (low homophily), the picture changes. The **GCN** model’s attack AUC rises to about **0.56** (medium density) and **0.59** (sparse)—meaning the attacker can guess membership better than random. In the same settings, logistic regression, MLP, and GraphSAGE stay around 0.51–0.53 (still near random). So **only GCN becomes clearly more vulnerable** when the graph structure is unfavorable; GraphSAGE behaves more like the non-graph models.

Numerical snapshot (no defense, confidence attack AUC):

Graph type (homophily / density)	LogReg	MLP	GCN	GraphSAGE
High / dense	0.485	0.512	0.494	0.509
High / medium	0.526	0.515	0.487	0.526
High / sparse	0.532	0.535	0.528	0.539
Low / dense	0.479	0.517	0.511	0.510
Low / medium	0.535	0.547	0.563	0.534
Low / sparse	0.508	0.519	0.592	0.520

Takeaway for the paper: Vulnerability is **model- and structure-dependent**. The claim “GNNs leak more than baselines” is not supported in our experiments; instead, “GCN leaks more when the graph has low homophily and is sparse” is supported.

4.2 RQ2: How Do Homophily and Density Affect Leakage?

Short answer: Low homophily and sparsity increase leakage for GCN. For GraphSAGE and baselines, the effect is smaller. When the model generalizes almost perfectly (high homophily), leakage stays near 0.5 regardless of density.

Homophily, in practice:

- **High homophily (~0.80):** In all our experiments (sparse, medium, dense), attack AUC stayed between about 0.49 and 0.54 for every model. So when “birds of a feather flock together,” the model does well, and there is little extra signal for the attacker to exploit.
- **Low homophily (~0.30):** For **GCN**, attack AUC increased from about 0.51 (low homophily + dense) to about 0.56 (low + medium) to **about 0.59** (low + sparse). So as the graph became both less “alike” and sparser, GCN leaked more. For **GraphSAGE** and the baselines, AUC stayed in the 0.51–0.55 range.

Density, in practice:

- For high homophily, making the graph sparser (fewer edges) gave a small increase in leakage (e.g., GCN from 0.494 to 0.528).
- For low homophily, the main driver was homophily; density added a smaller effect (sparse was still worst for GCN).

Why this makes sense (layman’s view):

When the graph is “friendly” (high homophily), the model learns clean patterns and generalizes well. Its behavior on training vs test data is similar, so the attacker has little to go on (attack AUC ~ 0.5). When the graph is “unfriendly” (low homophily) and sparse, GCN has a harder time; it may *memorize* training nodes more and behave differently on them, which is exactly what the attacker uses. So **leakage is tied to how well the model generalizes**, not just to “using the graph.”

4.3 RQ3: Do Lightweight Defenses Reduce Leakage?

Short answer: No defense helps in every setting. Label smoothing often *increases* leakage and hurts calibration. Edge sparsification is the only defense that gave a statistically significant privacy improvement in our study (for GCN on low-homophily medium-density graphs).

Label smoothing:

We found that **label smoothing made things worse** in several cases. For example, for GraphSAGE on low-homophily sparse graphs, attack AUC went from about 0.52 (no defense) to **about 0.61** (with label smoothing)—a large increase in leakage. Calibration error (ECE) also jumped (e.g., from near 0 to about 0.09–0.10). So the model became more overconfident in a way the attacker could use. **We do not recommend label smoothing as a privacy defense;** in our experiments it often increased leakage.

Edge sparsification:

For **GCN on low-homophily medium-density graphs**, attack AUC went from about **0.563** (no defense) to **0.538** (with edge sparsification). A paired t-test gave $p \sim 0.0015$, so this improvement is **statistically significant**. In other settings, edge sparsification had small or mixed effects, but it is the only defense that clearly *reduced* leakage in our tests.

DropEdge:

For GCN on low-homophily medium graphs, there was a small improvement (AUC 0.563 $\rightarrow$ 0.540, $p \sim 0.055$). In other settings, DropEdge sometimes slightly *increased* leakage (e.g., GCN high sparse). So it is **setting-dependent** and not a reliable privacy fix overall.

Early stopping and confidence masking:

Early stopping often gave almost the same attack AUC as no defense (no meaningful change). Confidence masking had small ups and downs and no clear pattern. So we did not see a consistent privacy benefit from these two.

Summary table (statistically significant effects only):

Setting (dataset, model)	Defense	Effect on attack AUC	p-value	Interpretation
Low homophily, medium, GCN	Edge sparsification	Decrease (0.563- >0.538)	~ 0.0015	Defense helps
Low homophily, sparse, GraphSAGE	Label smoothing	Increase (0.52->0.61)	~ 0.042	Defense hurts

Takeaway: Practitioners should **avoid label smoothing** if the goal is privacy, and **consider edge sparsification** when using GCN on graphs with low homophily and medium density.

4.4 RQ4: Utility–Privacy Tradeoff

Short answer: The best tradeoff (high accuracy, low leakage) happens when the model generalizes well—e.g., high homophily or GraphSAGE on low-homophily graphs. The worst is GCN on low-homophily sparse graphs. Edge sparsification can improve the tradeoff where it reduces AUC without much accuracy loss; label smoothing worsens it.

In plain language:

We want two things: the model should be *accurate* (utility) and *not leak membership* (privacy).

- **Best regions:** When the graph has high homophily (any density), or when we use GraphSAGE on low-homophily graphs, we get accuracy around 99–100% and attack AUC around 0.50–0.53—so **high utility and low leakage**.
- **Worst region:** GCN on low-homophily *sparse* graphs: accuracy drops to about 75–78% and attack AUC rises to about 0.59—**lower utility and higher leakage**.
- **Defenses:** Applying **edge sparsification** to GCN on low-homophily medium graphs reduced leakage with only a small accuracy impact—a **better tradeoff**. Applying **label smoothing** often increased leakage without improving accuracy—a **worse tradeoff**.

So the “frontier” of utility vs privacy is shaped by (1) **which model** and **which graph** you use, and (2) **which defense** you choose. Avoiding label smoothing and using edge sparsification where it helps (e.g., GCN on adverse structure) improves that frontier.

4.5 Shadow-Model Attack and Calibration

Shadow attack: Our shadow-model attack (train attacker on a separate “shadow” model, test on the target) agreed with the confidence-based attack: when confidence-based AUC was high, shadow AUC was high; when it was near 0.5, shadow was near 0.5. So our conclusions are not specific to one attack type.

Calibration (ECE): When accuracy was high and we did *not* use label smoothing, ECE was near 0. When we used label smoothing, ECE increased (e.g., to about 0.09–0.11). GCN on low-homophily graphs also had higher ECE (about 0.05–0.10) even without label smoothing. So **higher calibration error tended to go with higher leakage**—consistent with the idea that overconfident, poorly calibrated predictions give the attacker a stronger signal.

5. Discussion

5.1 Why Homophily and Density Matter (Plain Language)

Homophily tells us whether the graph “helps” or “hurts” the model. When it’s high, neighbors are good predictors; the model generalizes and leaks little. When it’s low, the graph is noisier; GCN in particular seems to rely on structure and can overfit or miscalibrate, which the attacker exploits. **Density** (how many links there are) modulates this: sparse + low homophily is the hardest setting for GCN and where we see the most leakage.

5.2 Why Label Smoothing Backfires

Label smoothing is often used to reduce overconfidence. In our experiments it *changed* the pattern of confidence in a way that made it *easier* for the attack classifier to tell members from non-members (e.g., a characteristic “smoothed” shape that differs between train and test). So it is **not** a safe privacy defense here and can be harmful.

5.3 Limitations

- All reported results are on **synthetic** graphs. Real citation or healthcare graphs may behave differently; we intend to include Cora/Citeseer (and optional real data) in future runs.
- We used confidence-based and shadow-model attacks; stronger or adaptive attacks might show different effects.
- We did not evaluate differential privacy or other heavy-weight defenses; we focused on lightweight, easy-to-deploy options.

6. Conclusion

We ran a systematic study of membership inference against graph and non-graph models on synthetic graphs with controlled **homophily** (“same-label connectivity”) and **density** (how many edges), and tested five lightweight defenses. Main conclusions:

1. **GNNs are not uniformly more vulnerable** than non-graph baselines; **GCN** is more vulnerable when the graph has **low homophily** and is **sparse**, while GraphSAGE and baselines remain closer to random guessing.
2. **Homophily and density** strongly influence leakage: high homophily keeps leakage near chance; low homophily and sparsity increase leakage, especially for GCN.
3. **Label smoothing often increases leakage** and should not be used as a privacy defense; **edge sparsification** is the only defense that significantly reduced leakage in our experiments (GCN, low-homophily medium-density).
4. **Vulnerability tracks generalization and calibration:** when the model generalizes well and is well calibrated, attack AUC is near 0.5; when it struggles (e.g., GCN on adverse structure), both ECE and leakage rise.

We provide a reproducible, config-driven framework and recommend that practitioners (a) choose models and interpret results in light of graph structure

(homophily and density), (b) avoid label smoothing for privacy, and (c) consider edge sparsification when using GCN on graphs with low homophily.

References

1. Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). Membership inference attacks against machine learning models. *IEEE S&P*.
 2. He, X., et al. (2021). Node-level membership inference attacks against graph neural networks. *arXiv:2102.05429*.
 3. Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *ICLR*.
 4. Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. *NeurIPS*.
 5. Rong, Y., et al. (2020). DropEdge: Towards deep graph convolutional networks on node classification. *ICLR*.
-

Report generated from the PrivacyGNN experimental framework. Results based on 420 runs (6 synthetic datasets $\times$ 4 models $\times$ defense configurations $\times$ 5 seeds).